Guided Practice: Angular Integration

Outcome

You will create a Node and Express web application that uses Angular as a client-side scripting framework to provide a user interface to create a ToDo list.

Resources Needed

- VCASTLE

-- OR --

- System configured for MEAN stack development.

Level of Difficulty

Moderate

Deliverables

Deliverables are marked in red font or with a red picture border around the screenshot. **Your name should be visible as directed in screenshots that you submit.**

You will submit a link to your GitHub repository that contains files associated with this assignment. This will include:

1. Screenshot(s) of the application running (when screenshots are needed is called out in various steps in the document).
2. Committed code files.

Program Specifications

This practice program is designed to demonstrate and help you gain a deeper understanding of creating and running a web application that uses Angular for client-side scripting. This guided practice is based on the Mozilla Developer Network Angular tutorial found at https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Angular_getting_started.

Development Steps

Create and Run an Angular Application

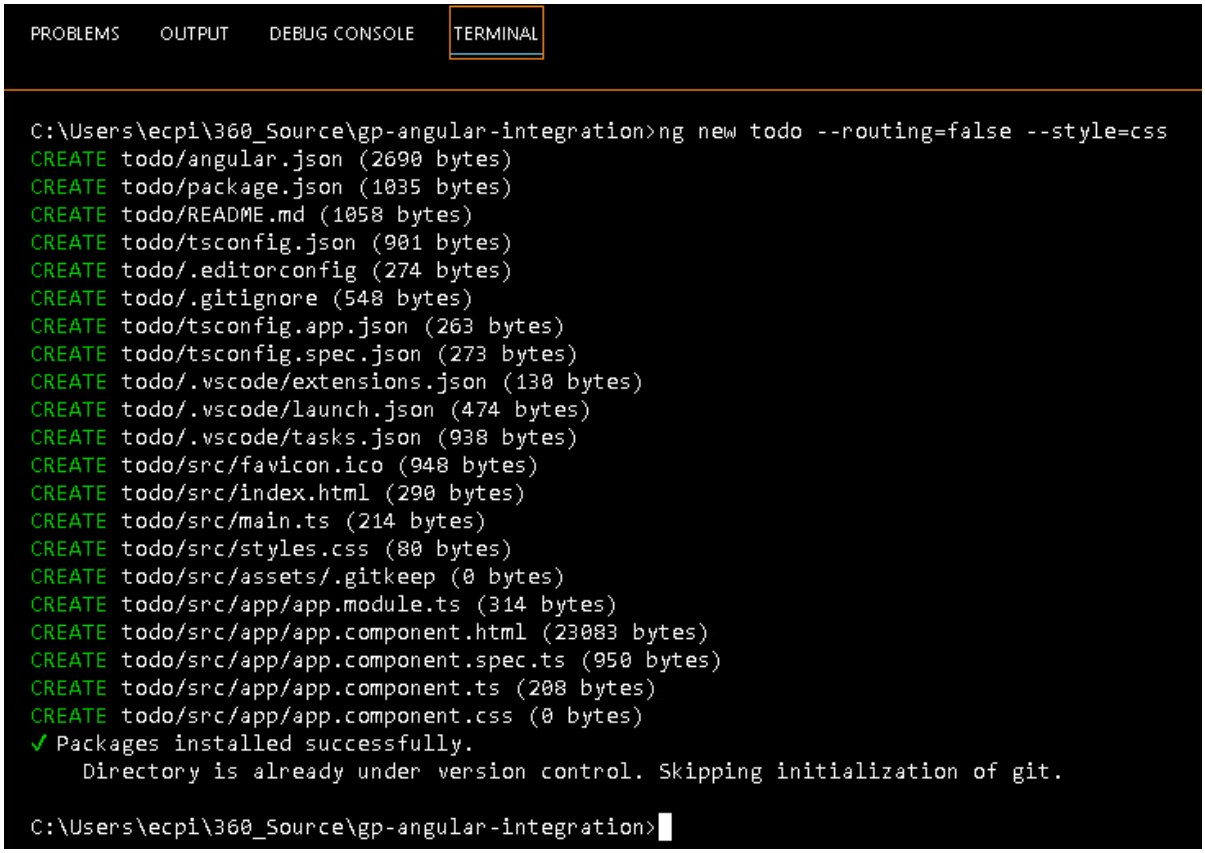
In this part of the guided practice, you will use the Angular Command Line Interface (Angular CLI) to create an application and make sure the code functions properly. This picks up where the guided practice instructions in Canvas leave off, which is with the project repository open in VS Code.

1. We're going to start by creating the Angular application using the Angular CLI:
    a. Open a command (cmd) terminal in VS Code.
    b. Run the command:
    ```
    ng new todo --routing=false --style=css
    ```

    Angular CLI commands all start with ng, followed by what you'd like the CLI to do.

The ng new command creates a minimal starter Angular application on your Desktop. The additional flags, --routing and --style, define how to handle navigation and styles in the application. This tutorial describes these features later in more detail.

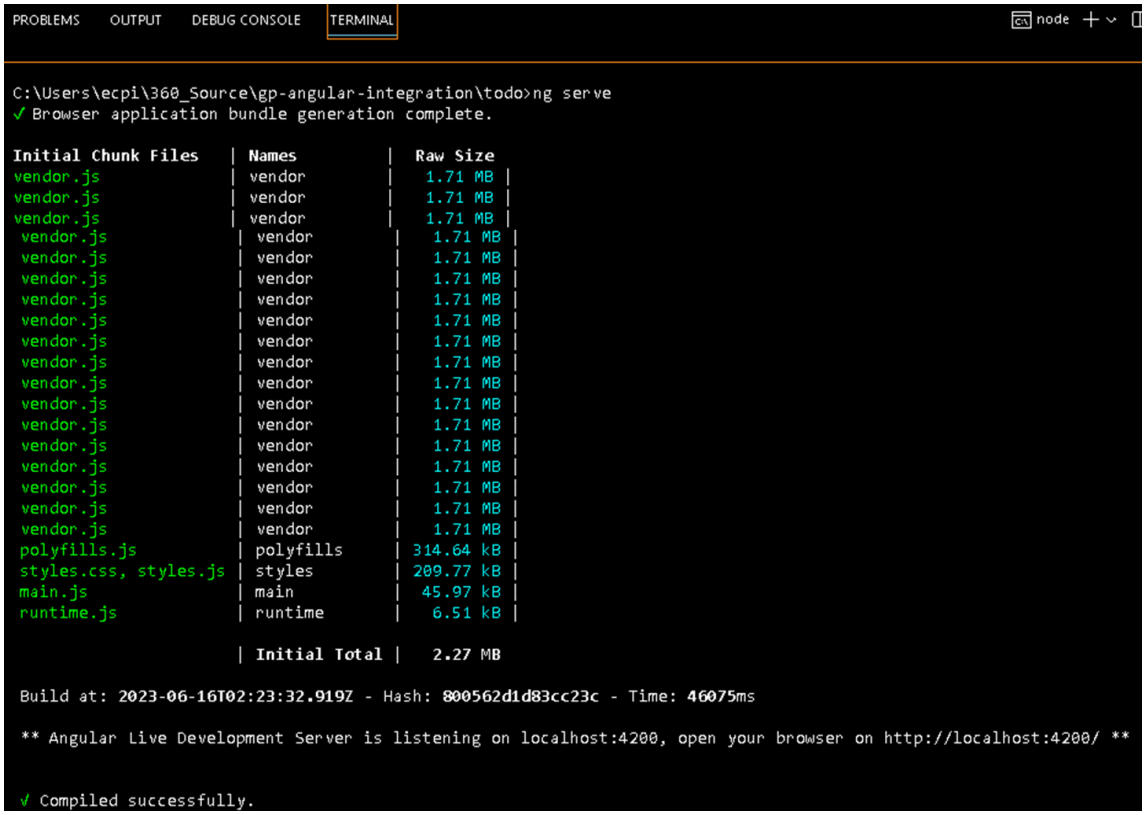
This should appear similar to the following:



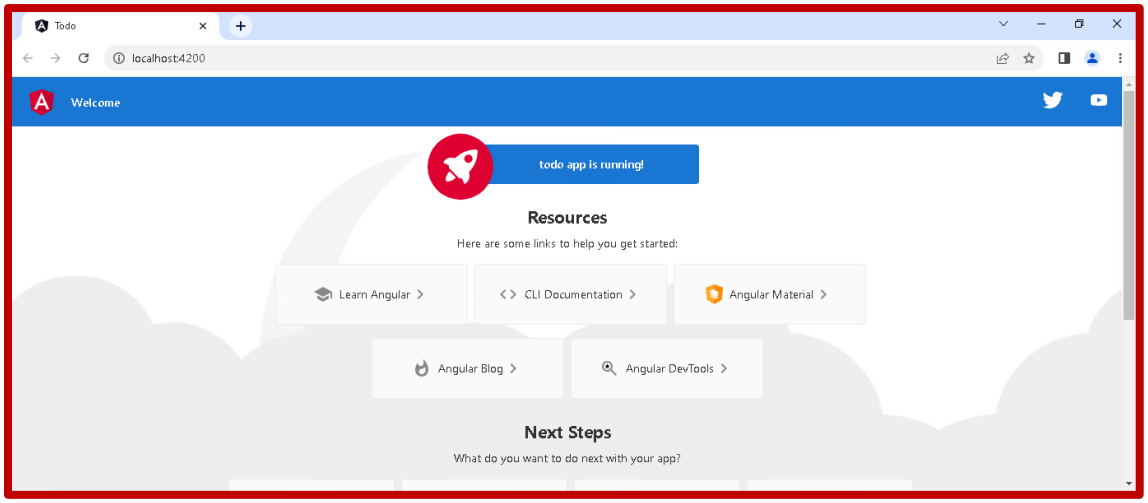
c. Once complete, Navigate into your new project with the following cd (change directory) command:
`cd todo`

d. Next, start the web server by running the command:
`ng serve`

This should result in something similar to the following:



e. Open a browser and navigate to http://localhost:4200/ to see your new starter application. If you change any of the source files, the application automatically reloads. The page should appear as follows:



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Understanding the Starter Code

The application source files that this tutorial focuses on are in src/app folder. The files the CLI generates automatically include the following:

1. app.module.ts: Specifies the files that the application uses. This file acts as a central hub for the other files in your application.
2. app.component.ts: Also known as the class, contains the logic for the application's main page.

3. app.component.html: Contains the HTML for AppComponent. The contents of this file are also known as the template. The template determines the view or what you see in the browser.
4. app.component.css: Contains the styles for AppComponent. You use this file when you want to define styles that only apply to a specific component, as opposed to your application overall.

A component in Angular is made up of three main parts—the template, styles, and the class. For example, app.component.ts, app.component.html, and app.component.css together constitute the AppComponent. This structure separates the logic, view, and styles so that the application is more maintainable and scalable, which is the best practice for web application development.

The Angular CLI also generates a file for component testing called app.component.spec.ts, but this tutorial doesn't go into testing, so you can ignore that file. Whenever you generate a component, the CLI creates these four files in a directory with the component name you specify.

Building the ToDo List App

At this point, we are ready to start creating our to-do list application using Angular. The finished application will display a list of to-do items and includes editing, deleting, and adding ToDo items. In this section we'll focus on understanding the application structure and displaying a basic list of ToDo items.

Just like a basic application that doesn't use a framework, an Angular application has an index.html. Within the <body> tag of the index.html, Angular uses a special element — <app-root> — to insert your main component, which in turn includes other components you create. Generally, you don't need to touch the index.html, instead focusing your work within specialized areas of your application called components.

The first directive we'll cover is Angular's iterator, *ngFor. *ngFor can dynamically create DOM elements based on items in an array. The second directive we'll learn is the logical operator, *ngIf. You can use *ngIf to add or remove elements from the DOM based on a condition.

1. The first thing to do is define the item object model, which defines what an item is. For our application, an item is an object that has a description and can be done. In the app (todo > src > app) directory, create a new file named item.ts with the following contents:

```
export interface Item {
  description: string;
  done: boolean;
}
```

2. Now that we've defined what an item is, we can give our application some items by adding them to the TypeScript file, app.component.ts. We'll also add the ability to add new items to our list. In app.component.ts, replace the contents with the following:

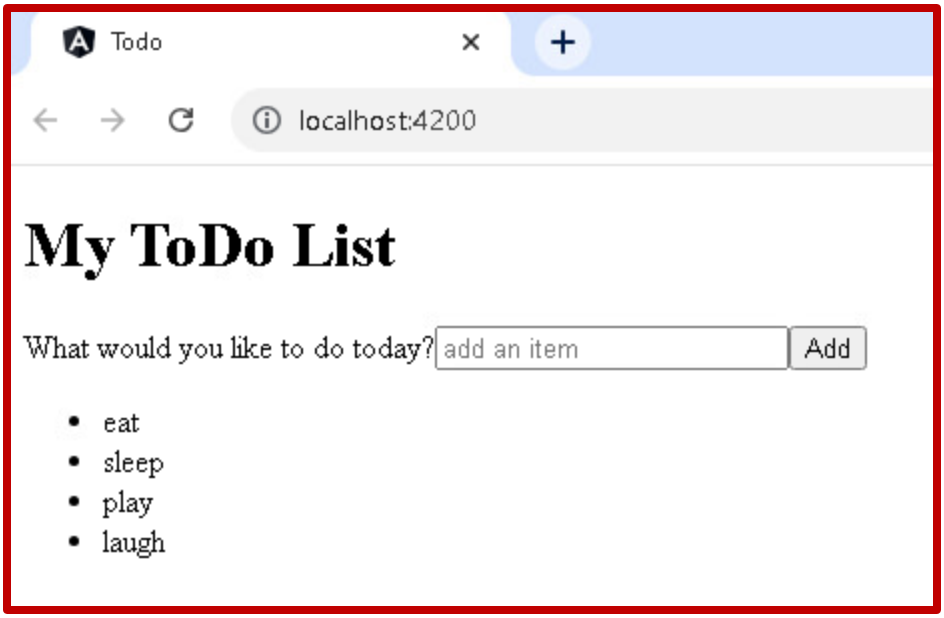
```
import { Component } from "@angular/core";

@Component({
  selector: "app-root",
  templateUrl: "./app.component.html",
  styleUrls: ["./app.component.css"],
})
export class AppComponent {
  title = "todo";

  filter: "all" | "active" | "done" = "all";
```

```
  allItems = [
    { description: "eat", done: true },
    { description: "sleep", done: false },
    { description: "play", done: false },
    { description: "laugh", done: false },
  ];

  get items() {
    if (this.filter === "all") {
      return this.allItems;
    }
    return this.allItems.filter((item) =>
      this.filter === "done" ? item.done : !item.done
    );
  }

  addItem(description: string) {
    this.allItems.unshift({
      description,
      done: false
    });
  }
}
```

3. Now we'll add the ability to display items in a list using the *ngFor iterator. To see the list of items in the browser, replace the contents of app.component.html with the following HTML:

```
<div class="main">
  <h1>My To Do List</h1>
  <label for="addItemInput">What would you like to do today?</label>

  <input
    #newItem
    placeholder="add an item"
    (keyup.enter)="addItem(newItem.value); newItem.value = ''"
    class="lg-text-input"
    id="addItemInput" />

  <button class="btn-primary" (click)="addItem(newItem.value)">Add</button>

  <ul>
    <li *ngFor="let item of items">{{item.description}}</li>
  </ul>
</div>
```

Save your changes and you'll see the application recompile, and your web page should now appear as follows. If you app isn't running, run it using the command ng serve and open a browser to localhost:4200.



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Adding some Style

The Angular CLI generates two types of style files:

- Component styles: The Angular CLI gives each component its own file for styles. The styles in this file apply only to its component.
- styles.css: In the src directory, the styles in this file apply to your entire application unless you specify styles at the component level.

1. First we'll add some styling for the app as a whole. In src > styles.css, add the following code:

```
body {
  font-family: Helvetica, Arial, sans-serif;
}

.btn-wrapper {
  /* flexbox */
  display: flex;
  flex-wrap: nowrap;
  justify-content: space-between;
}

.btn {
  color: #000;
  background-color: #fff;
  border: 2px solid #cecece;
  padding: 0.35rem 1rem 0.25rem 1rem;
  font-size: 1rem;
}

.btn:hover {
  background-color: #ecf2fd;
}
```

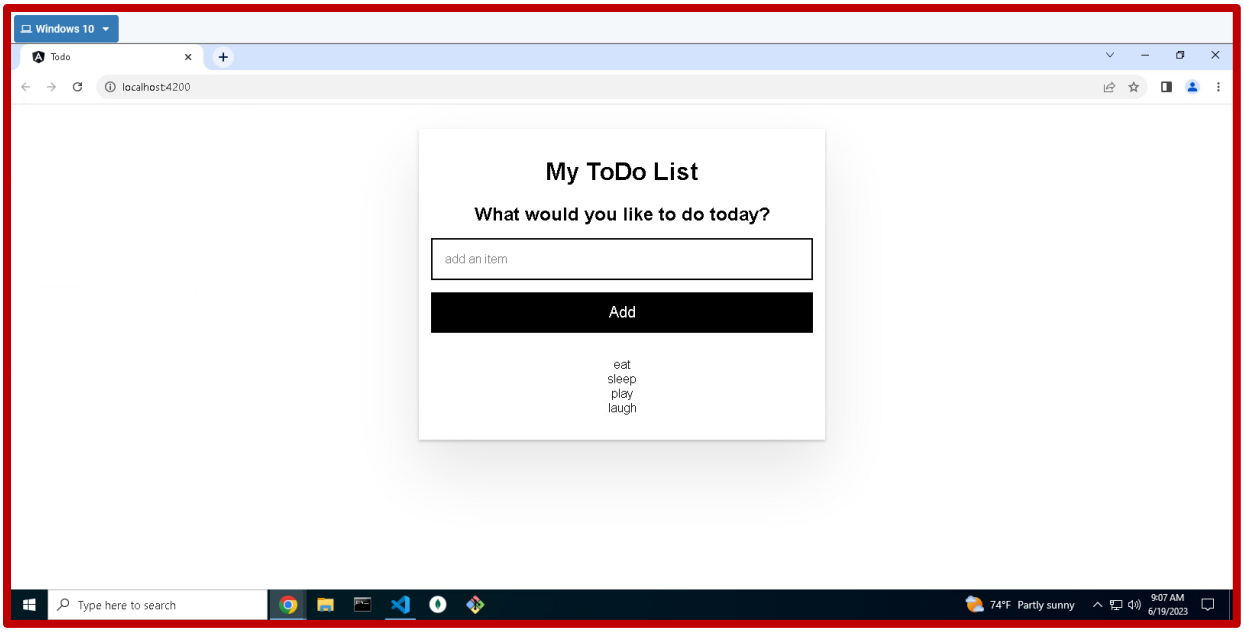
```
.btn:active {
  background-color: #d1e0fe;
}

.btn:focus {
  outline: none;
  border: black solid 2px;
}

.btn-primary {
  color: #fff;
  background-color: #000;
  width: 100%;
  padding: 0.75rem;
  font-size: 1.3rem;
  border: black solid 2px;
  margin: 1rem 0;
}

.btn-primary:hover {
  background-color: #444242;
}

.btn-primary:focus {
  color: #000;
  outline: none;
  border: #000 solid 2px;
  background-color: #d7ecff;
}

.btn-primary:active {
  background-color: #212020;
}
```

2. Next we'll add some specific styling for our component by adding the app.component.css file:

```
.main {
  max-width: 500px;
  width: 85%;
  margin: 2rem auto;
  padding: 1rem;
  text-align: center;
  box-shadow: 0 2px 4px 0 rgba(0, 0, 0, 0.2), 0 2.5rem 5rem 0 rgba(0, 0, 0, 0.1);
}

@media screen and (min-width: 600px) {
  .main {
    width: 70%;
  }
}
```

```
label {
  font-size: 1.5rem;
  font-weight: bold;
  display: block;
  padding-bottom: 1rem;
}

.lg-text-input {
  width: 100%;
  padding: 1rem;
  border: 2px solid #000;
  display: block;
  box-sizing: border-box;
  font-size: 1rem;
}

.btn-wrapper {
  margin-bottom: 2rem;
}

.btn-menu {
  flex-basis: 32%;
}

.active {
  color: green;
}

ul {
  padding-inline-start: 0;
}

ul li {
  list-style: none;
}
```

Save your changes and you'll see the application recompile, and your web page should now appear like the following:



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.

Adding an Item Component

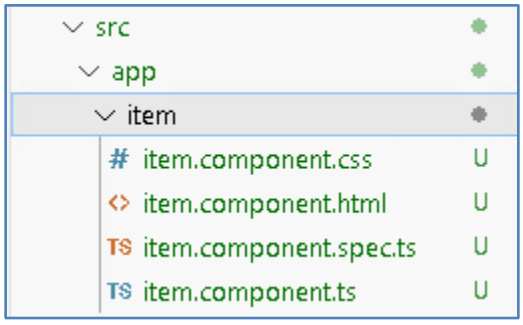
Components provide a way to organize an application. This section walks through creating a component to handle the individual items in the list and adding check, edit, and delete functionality. After all, what is a ToDo list if you can't check items off, update, and remove items from the list? We'll also cover the Angular event model in this section.

1. The first thing to do is create our component elements, which Angular provides tools for. Stop the Angular server by pressing Ctrl+C. To generate a component, run the command:
   `ng generate component item`

   This should look like the following:

```
C:\Users\ecpi\360_Source\gp-angular-integration\todo>ng generate component item
CREATE src/app/item/item.component.html (19 bytes)
CREATE src/app/item/item.component.spec.ts (585 bytes)
CREATE src/app/item/item.component.ts (194 bytes)
CREATE src/app/item/item.component.css (0 bytes)
UPDATE src/app/app.module.ts (388 bytes)

C:\Users\ecpi\360_Source\gp-angular-integration\todo>
```

   This creates the component files just like for the app.component. You'll see tht an "item" folder has been created and populated with 4 "item.component…" files:



2. Now that we've created the framework, we'll add HTML for the ItemComponent. Add the following code the the item.component.html file:

```
<div class="item">
  <input
    [id]="item.description"
    type="checkbox"
    (change)="item.done = !item.done"
    [checked]="item.done" />
  <label [for]="item.description">{{item.description}}</label>

  <div class="btn-wrapper" *ngIf="!editable">
    <button class="btn" (click)="editable = !editable">Edit</button>
    <button class="btn btn-warn" (click)="remove.emit()">Delete</button>
  </div>

  <!-- This section shows only if user clicks Edit button -->
  <div *ngIf="editable">
    <input
      class="sm-text-input"
      placeholder="edit item"
      [value]="item.description"
      #editedItem
      (keyup.enter)="saveItem(editedItem.value)" />

    <div class="btn-wrapper">
      <button class="btn" (click)="editable = !editable">Cancel</button>
      <button class="btn btn-save" (click)="saveItem(editedItem.value)">
        Save
      </button>
    </div>
  </div>
</div>
```

The first input is a checkbox so users can check off items when an item is complete. The double curly braces, {{}}, in the <label> for the checkbox signifies Angular's interpolation. Angular uses {{item.description}} to retrieve the description of the current item from the items array.

The next two buttons for editing and deleting the current item are within a . On this is an *ngIf, a built-in Angular directive that you can use to dynamically change the structure of the DOM. This *ngIf means that if editable is false, this is in the DOM. If editable is true, Angular removes this from the DOM.

When a user clicks the Edit button, editable becomes true, which removes this and its children from the DOM. If, instead of clicking Edit, a user clicks Delete, the ItemComponent raises an event that notifies the AppComponent of the deletion.

An *ngIf is also on the next , but is set to an editable value of true. In this case, if editable is true, Angular puts the and its child <input> and <button> elements in the DOM.

3. Now we'll add some logic to our ItemCompoent. First, add the following lines to the top (the import section) of the item.component.ts file (replace the existing import statement - these will be the only import statement needed):

```
import { Component, Input, Output, EventEmitter } from "@angular/core";
import { Item } from "../item";
```

Next we'll replace the generated ItemComponent class with the following:

```
export class ItemComponent {

  editable = false;

  @Input() item!: Item;
  @Output() remove = new EventEmitter<Item>();

  saveItem(description: string) {
    if (!description) return;
    this.editable = false;
    this.item.description = description;
  }
}
```

The editable property helps toggle a section of the template where a user can edit an item. editable is the same property in the HTML as in the *ngIf statement, *ngIf="editable". When you use a property in the template, you must also declare it in the class.

@Input(), @Output(), and EventEmitter facilitate communication between your two components. An @Input() serves as a doorway for data to come into the component, and an @Output() acts as a doorway for data to go out of the component. An @Output() has to be of type EventEmitter, so that a component can raise an event when there's data ready to share with another component.

The saveItem() method takes as an argument a description of type string. The description is the text that the user enters into the HTML <input> when editing an item in the list. This description is the same string from the <input> with the #editedItem template variable.

If the user doesn't enter a value but clicks Save, saveItem() returns nothing and does not update the description. If you didn't have this if statement, the user could click Save with nothing in the HTML <input>, and the description would become an empty string.

4. Now we'll wrap up our work on the item component by adding some style. Add the following to the item.component.css file:

```
.item {
  padding: 0.5rem 0 0.75rem 0;
  text-align: left;
  font-size: 1.2rem;
}

.btn-wrapper {
```

```
  margin-top: 1rem;
  margin-bottom: 0.5rem;
}

.btn {
  /* menu buttons flexbox styles */
  flex-basis: 49%;
}

.btn-save {
  background-color: #000;
  color: #fff;
  border-color: #000;
}

.btn-save:hover {
  background-color: #444242;
}

.btn-save:focus {
  background-color: #fff;
  color: #000;
}

.checkbox-wrapper {
  margin: 0.5rem 0;
}

.btn-warn {
  background-color: #b90000;
  color: #fff;
  border-color: #9a0000;
}

.btn-warn:hover {
  background-color: #9a0000;
}

.btn-warn:active {
  background-color: #e30000;
  border-color: #000;
}

.sm-text-input {
  width: 100%;
  padding: 0.5rem;
  border: 2px solid #555;
  display: block;
  box-sizing: border-box;
  font-size: 1rem;
```

```
  margin: 1rem 0;
}

/* Custom checkboxes
Adapted from https://css-tricks.com/the-checkbox-hack/#custom-designed-radio-
buttons-and-checkboxes */

/* Base for label styling */
[type="checkbox"]:not(:checked),
[type="checkbox"]:checked {
  position: absolute;
  left: -9999px;
}
[type="checkbox"]:not(:checked) + label,
[type="checkbox"]:checked + label {
  position: relative;
  padding-left: 1.95em;
  cursor: pointer;
}

/* checkbox aspect */
[type="checkbox"]:not(:checked) + label:before,
[type="checkbox"]:checked + label:before {
  content: "";
  position: absolute;
  left: 0;
  top: 0;
  width: 1.25em;
  height: 1.25em;
  border: 2px solid #ccc;
  background: #fff;
}

/* checked mark aspect */
[type="checkbox"]:not(:checked) + label:after,
[type="checkbox"]:checked + label:after {
  content: "\2713\0020";
  position: absolute;
  top: 0.15em;
  left: 0.22em;
  font-size: 1.3em;
  line-height: 0.8;
  color: #0d8dee;
  transition: all 0.2s;
  font-family: "Lucida Sans Unicode", "Arial Unicode MS", Arial;
}
/* checked mark aspect changes */
[type="checkbox"]:not(:checked) + label:after {
  opacity: 0;
  transform: scale(0);
```

```
}
[type="checkbox"]:checked + label:after {
  opacity: 1;
  transform: scale(1);
}

/* accessibility */
[type="checkbox"]:checked:focus + label:before,
[type="checkbox"]:not(:checked):focus + label:before {
  border: 2px dotted blue;
}
```

5. Now that we have an ItemComponent with logic, we'll use it in our app.component. Add the following line near the top of the app.component.ts file. This will import the ItemComponent functionality into our main app component:

```
import { Item } from "./item";
```

We also need to hook up the delete functionality we created in the item component. To do this, add the following code below the "addItem" function in app.component.ts:

```
remove(item: Item) {
  this.allItems.splice(this.allItems.indexOf(item), 1);
}
```

6. Finally we'll update the HTML for our app component to use our item component. Replace the current unordered list (ul) in app.component.html with the following:

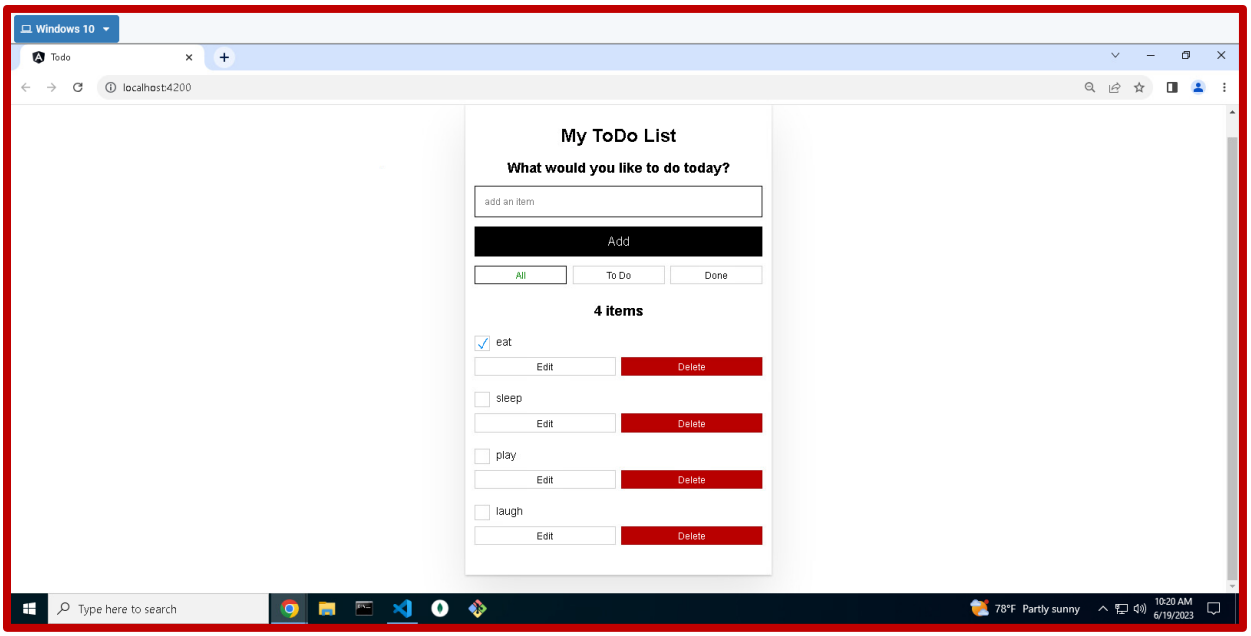
```
<h2>
  {{items.length}}
  <span *ngIf="items.length === 1; else elseBlock">item</span>
  <ng-template #elseBlock>items</ng-template>
</h2>

<ul>
  <li *ngFor="let i of items">
    <app-item (remove)="remove(i)" [item]="i"></app-item>
  </li>
</ul>
```

While we're in here, let's add the ability to filter our ToDo list. We already have the filter items set up in the app.component.ts file - we did that when we updated that class in the beginning, and we included logic there for filtering, so now we just need to add the display to hook up to it. Below the Add button code, add the following:

```
<!-- Buttons that show all, still to do, or done items on click -->
<div class="btn-wrapper">
```

```
  <button
    class="btn btn-menu"
    [class.active]="filter == 'all'"
    (click)="filter = 'all'">
    All
  </button>

  <button
    class="btn btn-menu"
    [class.active]="filter == 'active'"
    (click)="filter = 'active'">
    To Do
  </button>

  <button
    class="btn btn-menu"
    [class.active]="filter == 'done'"
    (click)="filter = 'done'">
    Done
  </button>
</div>
```

7. Save your changes and run the application using the command ng serve and open a browser to localhost:4200. (Note: remember that you need to run this command from the “todo” folder.)



Take a screenshot that resembles the one above and save it in the same folder as your README.md file in the repository.